

AI Assisted Coding – Lab Assignment 7.5

Lab 7: AI-Based Code Completion – Classes, Loops, and Conditionals

Name: K. Sai Krishna Reddy

HT.No: 2303A52305

Batch: 45

◆ Task 1: Mutable Default Argument – Function Bug

Task: Analyze given code where a mutable default argument causes unexpected behavior. Use AI to fix it.

Bug: Mutable default argument

```
def add_item(item, items=[]):  
    items.append(item)  
  
    return items  
  
print(add_item(1))  
  
print(add_item(2))
```

Expected Output: Corrected function avoids shared list bug.

Understanding the Problem

The function uses a mutable list (`items=[]`) as a default argument. In Python, default arguments are evaluated only once at function definition time, not every call. This causes the same list to be shared across function calls.

Why the Bug Occurs

When `add_item(1)` runs, list becomes `[1]`.

When `add_item(2)` runs, instead of creating a new list, it uses the same list → `[1,2]`.

AI-Assisted Fix Strategy

AI suggests replacing default argument with None and initializing inside function.

Prompt

Analyze the following Python function and fix the mutable default argument issue:

Code

```
def add_item(item, items=None):
```

```
    if items is None:
```

```
        items = []
```

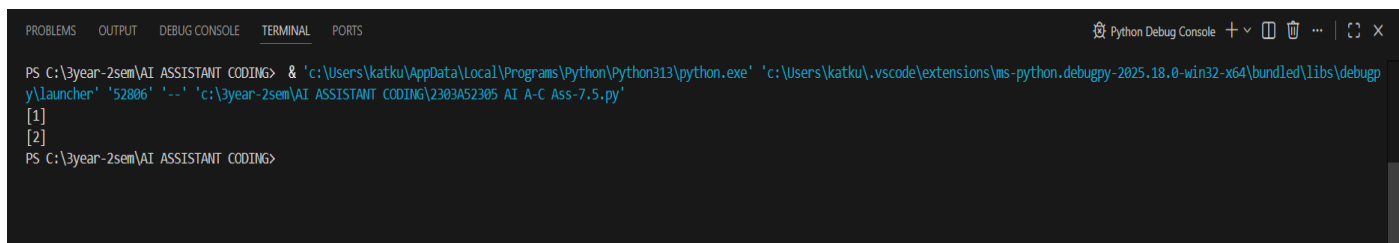
```
    items.append(item)
```

```
    return items
```

```
print(add_item(1))
```

```
print(add_item(2))
```

Output



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python Debug Console + v [ ] ... | [ ] X
PS C:\3year-2sem\AI ASSISTANT CODING> & 'c:\Users\katku\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\katku\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '52806' '--' 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-7.5.py'
[1]
[2]
PS C:\3year-2sem\AI ASSISTANT CODING>
```

Observation

- AI correctly identifies this as a common Python pitfall.
- Shows AI has knowledge of Python memory behavior.
- Demonstrates importance of understanding reference vs value behavior.

◆ Task 2: Floating-Point Precision Error

Task: Analyze given code where floating-point comparison fails.

Use AI to correct with tolerance.

Bug: Floating point precision issue

```
def check_sum():  
    return (0.1 + 0.2) == 0.3  
  
print(check_sum())
```

Expected Output: Corrected function

Understanding the Problem

Floating-point arithmetic in computers is approximate due to binary representation.

0.1 + 0.2 does not equal exactly 0.3.

Why the Bug Occurs

Binary floating-point cannot precisely represent decimal fractions.

AI-Assisted Fix Strategy

AI suggests comparing values within a tolerance using $\text{abs}(a - b) < \text{epsilon}$.

Prompt

Fix floating-point comparison issue using tolerance:

Code

```
def check_sum():  
    return abs((0.1 + 0.2) - 0.3) < 1e-9  
  
print(check_sum())
```

Output



```
PS C:\3year-2sem\AI ASSISTANT CODING>
PS C:\3year-2sem\AI ASSISTANT CODING>
PS C:\3year-2sem\AI ASSISTANT CODING> c: cd 'c:\3year-2sem\AI ASSISTANT CODING'; & 'c:\Users\katku\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\katku\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '59554' '-.' 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-7.5.py'
True
PS C:\3year-2sem\AI ASSISTANT CODING>
```

Observation

- AI recognizes numeric precision limitations.
- Suggests industry-standard tolerance comparison.
- Demonstrates awareness of computer architecture limitations.

◆ Task 3: Recursion Error – Missing Base Case

Task: Analyze given code where recursion runs infinitely due to missing base case. Use AI to fix.

Bug: No base case

```
def countdown(n):
    print(n)
    return countdown(n-1)
countdown(5)
```

Expected Output: Correct recursion with stopping condition.

Understanding the Problem

Recursive function countdown(n) calls itself endlessly because no stopping condition is defined.

Why the Bug Occurs

Without a base case, recursion continues until stack overflow.

AI-Assisted Fix Strategy

AI adds base condition like if $n \leq 0$: return.

Prompt

Fix recursion function to include proper stopping condition:

Code

```
def countdown(n):
    if n < 0:
        return
    print(n)
    return countdown(n-1)
countdown(5)
```

Output

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Python Debug Console + v [ ] [ ] ... [ ] [ ] X

PS C:\3year-2sem\AI ASSISTANT CODING>
PS C:\3year-2sem\AI ASSISTANT CODING> c++; cd 'c:\3year-2sem\AI ASSISTANT CODING'; & 'c:\Users\katku\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\katku\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '56147' '--' 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-7.5.py'
5
4
3
2
1
0
PS C:\3year-2sem\AI ASSISTANT CODING>

```

Observation

- AI detects infinite recursion pattern quickly.
- Demonstrates AI's understanding of recursion principles.
- Reinforces importance of termination conditions.

◆ Task 4: Dictionary Key Error

Task: Analyze given code where a missing dictionary key causes error. Use AI to fix it.

Bug: Accessing non-existing key

```
def get_value():  
    data = {"a": 1, "b": 2}  
    return data["c"]  
    print(get_value())
```

Expected Output: Corrected with .get() or error handling.

Understanding the Problem

Accessing data["c"] when key does not exist causes KeyError.

Why the Bug Occurs

Dictionary does not contain key "c".

AI-Assisted Fix Strategy

AI suggests using .get() method or try-except block.

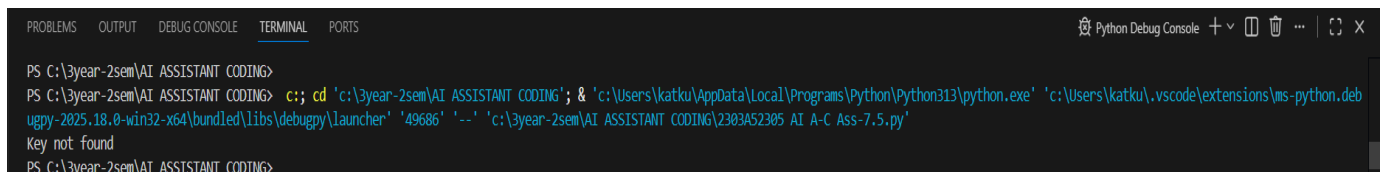
Prompt

Fix dictionary KeyError issue:

Code

```
def get_value():  
    data = {"a": 1, "b": 2}  
    return data.get("c", "Key not found")  
    print(get_value())
```

Output



The screenshot shows a terminal window with the following content:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  
Python Debug Console + v [icon] [icon] [icon] X  
PS C:\3year-2sem\AI ASSISTANT CODING>  
PS C:\3year-2sem\AI ASSISTANT CODING> cd 'c:\3year-2sem\AI ASSISTANT CODING'; & 'c:\Users\katku\AppData\Local\Programs\python\python313\python.exe' 'c:\Users\katku\.vscode\extensions\ms-python.de  
bugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '49686' '--' 'c:\3year-2sem\AI ASSISTANT CODING\2303452305 AI A-C Ass-7.5.py'  
Key not found  
PS C:\3year-2sem\AI ASSISTANT CODING>
```

Observation

- AI recognizes runtime error pattern.

- Provides safe dictionary access approach.
 - Encourages defensive programming.
-

◆ Task 5: Infinite Loop – Wrong Condition

Task: Analyze given code where loop never ends. Use AI to detect and fix it.

Bug: Infinite loop

```
def loop_example():
```

```
    i = 0
```

```
    while i < 5:
```

```
        print(i)
```

Expected Output: Corrected loop increments i.

Understanding the Problem

Variable i is never incremented, so loop condition remains true forever.

Why the Bug Occurs

Missing update statement inside loop.

AI-Assisted Fix Strategy

AI adds `i += 1` inside loop.

Prompt

Detect and fix infinite loop:

Code

```
def loop_example():
```

```
    i = 0
```

```
i += 1
```

loop_example()

Output

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Python Debug Console + v [trash] [refresh] [close]
PS C:\3year-2sem\AI ASSISTANT CODING>
PS C:\3year-2sem\AI ASSISTANT CODING> c:: cd 'c:\3year-2sem\AI ASSISTANT CODING'; & 'c:\Users\katku\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\katku\.vscode\extensions\ms-python.de
bugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '52045' '--' 'c:\3year-2sem\AI ASSISTANT CODING\2303452305 AI A-C Ass-7.5.py'
0
1
2
3
4
PS C:\3year-2sem\AI ASSISTANT CODING>
```

Observation

- AI detects control flow issue.
- Demonstrates understanding of loop mechanics.
- Highlights importance of state updates.

◆ Task 6: Unpacking Error – Wrong Variables

Task: Analyze given code where tuple unpacking fails. Use AI to fix it.

Bug: Wrong unpacking

$$a, b = (1, 2, 3)$$

Expected Output: Correct unpacking or using _ for extra values.

Understanding the Problem

Tuple has 3 values but only 2 variables are provided for unpacking.

Why the Bug Occurs

Mismatch between number of variables and tuple elements.

AI-Assisted Fix Strategy

AI suggests:

- Add third variable
- Or use underscore `_` for ignoring extra values

Prompt

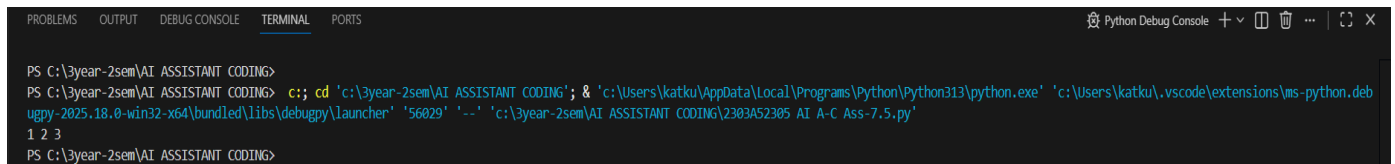
Fix tuple unpacking error:

Code

```
a, b, c = (1, 2, 3)
```

```
print(a,b,c)
```

Output



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python Debug Console + v [ ] ... [ ] X
PS C:\3year-2sem\AI ASSISTANT CODING>
PS C:\3year-2sem\AI ASSISTANT CODING> c++; cd 'c:\3year-2sem\AI ASSISTANT CODING'; & 'c:\Users\katku\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\katku\.vscode\extensions\ms-python.de
ugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '56029' '-.' 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-7.5.py'
1 2 3
PS C:\3year-2sem\AI ASSISTANT CODING>
```

Observation

- AI understands Python unpacking rules.
- Suggests clean syntax correction.
- Encourages readable unpacking patterns.

◆ Task 7: Mixed Indentation – Tabs vs Spaces

Task: Analyze given code where mixed indentation breaks execution. Use AI to fix it.

Bug: Mixed indentation

```
def func():
```

```
    x = 5
```

```
    y = 10
```

```
    return x+y
```

Expected Output: Consistent indentation applied.

Understanding the Problem

Python enforces consistent indentation. Mixed tabs and spaces cause syntax errors.

Why the Bug Occurs

Inconsistent indentation levels break block structure.

AI-Assisted Fix Strategy

AI reformats code using consistent indentation (4 spaces).

Prompt

Fix indentation error:

Code

```
def func():
```

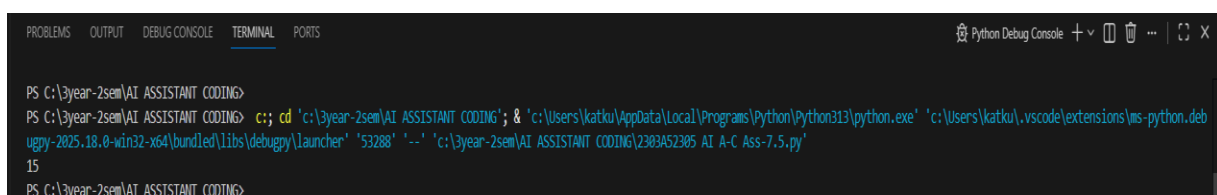
```
    x = 5
```

```
    y = 10
```

```
    return x + y
```

```
print(func())
```

Output



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python Debug Console + - [ ] [ ] [ ] X
PS C:\3year-2sem\AI ASSISTANT CODING>
PS C:\3year-2sem\AI ASSISTANT CODING> cd 'c:\3year-2sem\AI ASSISTANT CODING'; & 'c:\Users\katku\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\katku\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '53288' '--' 'c:\3year-2sem\AI ASSISTANT CODING\2303AS2305 AI A-C Ass-7.5.py'
15
PS C:\3year-2sem\AI ASSISTANT CODING>
```

Observation

- AI easily detects formatting errors.
 - Demonstrates syntax-level debugging ability.
 - Highlights importance of code formatting standards.
-

◆ Task 8: Import Error – Wrong Module Usage

Task: Analyze given code with incorrect import. Use AI to fix.

Bug: Wrong import

```
import maths
```

```
print(maths.sqrt(16))
```

Expected Output: Corrected to import math

Understanding the Problem

maths module does not exist in Python standard library.

Why the Bug Occurs

Incorrect module name.

AI-Assisted Fix Strategy

AI corrects import to import math.

Prompt

Fix incorrect module import:

Code

```
import math
```

```
print(math.sqrt(16))
```

Output

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Python Debug Console + ▾ 🛑 ... | 🔍 ✕

```
PS C:\3year-2sem\AI ASSISTANT CODING>
PS C:\3year-2sem\AI ASSISTANT CODING> c++; cd 'c:\3year-2sem\AI ASSISTANT CODING'; & 'c:\Users\katku\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\katku\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '58331' '--' 'c:\3year-2sem\AI ASSISTANT CODING\2383A52305 AI A-C Ass-7.5.py'
4.0
PS C:\3year-2sem\AI ASSISTANT CODING>
```

Observation

- AI identifies incorrect library usage.
- Shows awareness of Python standard modules.
- Helps avoid trivial but common mistakes.

Overall Lab Observations

Error Type	AI Strength	Human Role
Syntax Errors	Quickly detected	Verify correctness
Logic Errors	Identifies patterns	Validate intent
Runtime Errors	Suggests safe methods	Test edge cases
Recursion Issues	Adds base case	Ensure termination
Structural Bugs	Reformats properly	Maintain standards